

Lab03

Mateus Bortolucci

1. Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.

Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE; Para o arquivo de vôos: Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY; Leia apenas 1 milhão de linhas por vez; Remova vôos em que o aeroporto de destino comece com a letra 1; Remova registros em que existam pelo menos uma coluna faltante; Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino; Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0
—
✓ dplyr      1.2.1    ✓ readr      2.2.0
✓ forcats    1.0.1    ✓ stringr    1.6.0
✓ ggplot2    4.0.3    ✓ tibble     3.3.1
✓ lubridate  1.9.5    ✓ tidyr      1.3.2
✓ purrr      1.2.2
— Conflicts — tidyverse_conflicts()
—
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```
library(leaflet)
library(webshot2)

Sys.setenv(CHROMOTE_CHROME_ARGS = "--no-sandbox --disable-dev-shm-usage --
disable-gpu")

airlines <- read_csv("airlines.csv")
```

```
Rows: 14 Columns: 2
— Column specification
```

```
Delimiter: ","
chr (2): IATA_CODE, AIRLINE
```

```
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this
message.
```

```
airports <- read_csv(
  "airports.csv",
  col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE)
)
```

```
Rows: 322 Columns: 5
— Column specification
```

```
Delimiter: ","
chr (3): IATA_CODE, CITY, STATE
dbl (2): LATITUDE, LONGITUDE
```

```
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this
message.
```

```
processa_lote <- function(df, pos) {
  df %>%
    filter(!str_starts(DESTINATION_AIRPORT, "^1")) %>%
    drop_na() %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      soma_atraso = sum(ARRIVAL_DELAY),
      qtd_voos = n(),
      .groups = "drop"
    )
}

colunas_interesse <- cols_only(
  DESTINATION_AIRPORT = col_character(),
  ARRIVAL_DELAY = col_double()
)

estatisticas_lotes <- read_csv_chunked(
  file = "flights.csv",
  callback = DataFrameCallback$new(processa_lote),
  chunk_size = 1000000,
  col_types = colunas_interesse
)
```

```

)

flights <- estatisticas_lotes %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(
    total_atraso = sum(soma_atraso),
    total_voos = sum(qtd_voos),
    atraso_medio = total_atraso / total_voos,
    .groups = "drop"
  )

```

2. Selecione a operação apropriada join para incluir, na tabela flights, as colunas CITY e STATE do objeto airports. Para executar esta tarefa:

Identifique a coluna que é a chave na tabela flights; Identifique a coluna que é a chave na tabela airports; Quais são os aeroportos que estão listados em flights, mas estão ausentes em airports? Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves; Armazene a tabela resultante no objeto flights.

```

aeroportos_ausentes <- anti_join(
  flights,
  airports,
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)
print(aeroportos_ausentes)

```

```

# A tibble: 0 × 4
#   i 4 variables: DESTINATION_AIRPORT <chr>, total_atraso <dbl>,
#   total_voos <int>, atraso_medio <dbl>

```

```

flights <- left_join(
  flights,
  airports,
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)

```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```

tabela_estados <- airports %>%
  count(STATE, name = "numero_aeroportos") %>%
  arrange(desc(numero_aeroportos))

print(tabela_estados)

```

```
# A tibble: 54 × 2
  STATE numero_aeroportos
  <chr>          <int>
1 TX              24
2 CA              22
3 AK              19
4 FL              17
5 MI              15
6 NY              14
7 CO              10
8 MN               8
9 MT               8
10 NC              8
# i 44 more rows
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

Carregue o pacote leaflet; Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico; Armazene o grafico em b) numa variável chamada this; Adicione as duas linhas abaixo após a criação da variável this (apenas se você estiver usando Jupyter):

```
flights_mapa <- flights %>%
  drop_na(LATITUDE, LONGITUDE)

this <- leaflet(data = flights_mapa) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~paste("Aeroporto:", DESTINATION_AIRPORT, "<br>",
                  "Atraso Médio:", round(atraso_medio, 2), "min")
  )
```

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-09-01 00:44:29 -03"
```